Marking Scheme

Module Code: COMP I8064

Module Title: Data Structures

Internal Examiner(s): Dr Michelle Graham

Session: Winter

Year: 25-26

Instructions:

Answer QUESTION ONE [40 marks] and any other TWO questions [30 marks each].

Marking Scheme

Marking Scheme

You MUST attempt this question.

Question 1) Data Structure Fundamentals

(a) Data structures are designed to store and operate on data within a program.

(i) What is an Abstract Data Type (ADT)? What is the benefit of separating a data structure's ADT from its implementation?

(5 marks)

(ii) Formally specify the ADT for a *List* structure. Your ADT should include all fundamental operations for a *List* and explain the need for their presence.

(10 marks)

(b) Generics are a programming feature designed to make data structures more flexible and reusable.

(i) Explain what is meant by a generic data structure and how it differs from a non-generic data structure. Provide an example to illustrate your answer.

(4 marks)

KEY ASPECTS:

Must mention the ability to hold any type of data vs a fixed type.

Must mention reusability.

MARKS BREAKDOWN:

1 mark - Definition

2 marks - Explanation

1 mark - Example

Marking Scheme

(ii) Outline and explain the key code-level elements required to implement a generic data structure in Java. In your answer you should refer to both **node-based** structures (e.g. linked lists) and **array-based** structures (e.g. dynamic arays).

(5 marks)

KEY ASPECTS:

Type variable (<T>) at top of class

Returning T type instead of fixed type

Taking T type instead of fixed type as parameter

Node specific:

- Node class also containing type variable on class signature
- Node class storing T type instead of fixed type
- Node class taking in T type as parameter instead of fixed type

Array-based structures:

- Storing data in Object array
- Casting data to type variable type as it is retrieved from array
- Suppressing warnings on retrieval action code

MARKS BREAKDOWN:

2 marks - General (node and array-based) elements

1.5 marks - Node-specific elements

1.5 marks - Array-based elements

(iii) Explain one advantage and one potential drawback to implementing a data structure as a **generic** version vs implementing as a **type-specific** (non-generic) version.

(4 marks)

KEY ASPECTS:

Advantages:

- Reusability
- Reduced code duplication

Drawbacks:

- Increased complexity
- Overheads associated with autoboxing primitives

Marking Scheme

- Slightly more expensive to execute due to type erasure costing more in checks
- Less control/less opportunity for optimisation

MARKS BREAKDOWN:

Benefit: 2 marks

Drawback: 2 marks

(c) Your team has been asked to write unit tests for a PriorityQueue. The PriorityQueue is built to hold Task objects, is unbounded, and duplicate values are allowed. Null values should not be allowed. The Task class contains a compareTo method that ranks instances in order of priority (with highest being 5 and lowest being 1).

You have been assigned the following method:

- public boolean add(String s)

Outline **three** test cases that should be included as unit tests for the above method. For each test case, explain its necessity, what assertions should be included and why they are needed. You may assume the existence of a get() method to allow direct access to specific points in the PriorityQueue

(12 marks)

SAMPLE ANSWER:

Adding null (throws exception y/n) – confirms null values are rejected correctly

Adding Task of equal priority (assert size increases, assert queue contains all elements it should contain in correct order, assert new task is added AFTER element with equal priority) – Confirms equal priority with be added AFTER existing task in queue.

Adding duplicate Task - (assert size increases, assert queue contains all elements it should contain in correct order, assert new task is added AFTER element with equal priority) – Confirms duplicate element is not rejected and is added in after original one.

Marking Scheme

MARKS BREAKDOWN:

Test case explanation:	1 mark * 3
Test case necessity:	1 mark * 3
Assertions included	1 mark * 3
Necessity of assertions	1 mark * 3

Question 2) Stacks and Queues

(a) Explain the concept of "fail fast" programming and how it can be applied to the implementation of a **Stack** data structure. Include one or more examples of how a Stack can detect and respond to invalid operations at runtime.

(5 marks)

KEY ASPECTS:

Fail early - Example: check if stack is empty immediately, don't attempt to do anything else first

Fail visibly - Example: throw an exception if stack is empty when pop is called

Rationale: Better for debugging, faster execution, any other reasonable logic.

MARKS BREAKDOWN:

Fail early:	1 mark
Fail visibly:	1 mark
Rationale:	1 mark
Examples:	2 marks

(b) A PriorityQueue is a structure in which the data is stored in order of its relative priority.

(i) Write and explain the **pseudocode** for an *add(Job j)* method that inserts a Job object into a PriorityQueue implemented as a **singly-linked node-based List**.

Each Job has three attributes:

Marking Scheme

- priority - an integer between 1 & 5 (5 is highest priority)
- description - a string describing the Job
- timestamp – the time when the Job was created (a LocalDateTime)

The PriorityQueue should stay sorted in descending order of Job priority (highest priority first). Your answer should include pseudocode for the insertion logic, as well as an explanation of what each step does and why it is needed.

*Note: Your method should be done as **pseudocode**, not actual code. Pseudocode and explanation are worth equal marks.*

(15 marks)

MARKS BREAKDOWN:

6 marks	-	Pseudocode
6 marks	-	Explanation
3 marks	-	Consideration of equal priorities

(ii) Evaluate whether a singly-linked node-based List is an efficient structure for implementing a PriorityQueue. If not, suggest a more suitable alternative and justify your choice with reference to worst-case performance and operation efficiency.

(10 marks)

MARKS BREAKDOWN:

1 mark	-	Verdict (should be no)
4 marks	-	Explanation
1 mark	-	Alternative recommendation
4 marks	-	Justification

Question 3) Binary Search Trees

Marking Scheme

(a) Explain the concept and structure of the *Binary Search Tree (BST)* data structure. List and describe the fundamental operations that can be performed on a *Binary Search Tree*, outlining their general purpose and time complexity.

(8 marks)

KEY ASPECTS:

BST explanation:

- Tree of nodes with left and right branches
- Nodes are inserted based on their value (larger to the right, smaller to the left)
- Insertion maintains sorted nature of data
- Data must be orderable (Comparable) to facilitate ordered insertion

Key operations:

- Add
- Remove
- Contains
- Min
- Max
- iterator

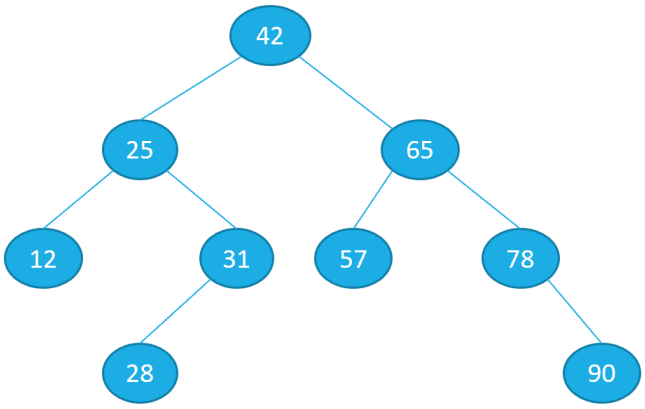
MARKS BREAKDOWN:

3 marks - Binary Search Tree explanation

2 marks - List of operations

3 marks - Explanation of operations

(b) The following is a depiction of a Binary Search Tree (BST).



Marking Scheme

(i) What is meant by the *traversal* of a BST tree? Explain the three orders of tree traversal, illustrating your answer with the result of the tree's traversal under each approach.

(14 marks)

KEY COMPONENTS:

Tree traversal: process of visiting every node of a tree **exactly once** in a systematic, predefined order. Traversal allows us to examine or process each node and is fundamental for tasks such as searching, printing, or evaluating tree structures.

We can traverse trees in three main ways:

- In-order
- Pre-order
- Post-order

In-order traversal examines the node and its children going from left to right

- left child -> current node -> right child
- Tree traversal: 12, 25, 28, 31, 42, 57, 65, 78, 90

Pre-order traversal examines the current node BEFORE looking at its children

- current node -> left child -> right child
- Tree traversal: 42, 25, 12, 31, 28, 65, 57, 78, 90

Post-order traversal examines the current node AFTER dealing with its children

- left child -> right child -> current node
- Tree traversal: 12, 28, 31, 25, 57, 90, 78, 65, 42

MARKS BREAKDOWN:

2 marks -> Traversal explanation
2 marks * 3 -> Order explanation
2 marks * 3 -> traversal of tree

Marking Scheme

(ii) Explain what is meant by an **unbalanced** Binary Search Tree (BST). In your answer, describe one scenario in which a BST may become unbalanced, and discuss how this imbalance affects the performance of common BST operations such as search, insertion, and deletion. Your explanation should include reference to Big-O time complexity.

(8 marks)

KEY COMPONENTS:

Unbalanced tree: Tree where one subtree is significantly deeper than the other OR height is far from optimal.

Example scenarios causing unbalanced tree:

- Inserting sorted or nearly sorted data
- Sequential inserts creating a chain
- Repeated skewed insertions to one side

Impact on big O:

- Balanced tree actions: O(log n)
- Unbalanced tree: O(n)
- Explanation: tree becomes like a linked list – need to travel through all elements in sequence

MARKS BREAKDOWN:

2 marks -> Definition

3 marks -> Scenario explanation

3 marks -> Impact on Big O

Question 4) Analysis of Data Structures

A company operates a real-time analytics platform that collects millions of event records per hour. Each event has:

- A unique eventID (String)
- A timestamp (LocalDateTime)

Marking Scheme

- Associated metadata (key–value information)

The system must:

- Efficiently **store** and **access** these event records.
- Support **fast existence checks** to see if a given eventID has already been processed.
- Occasionally retrieve events in **sorted order** by eventID for reporting.
- Use **minimal memory** and minimize performance loss caused by hash collisions.

The development team is considering three data structures:

- **HashSet**
- **TreeSet** (implemented as a *Binary Search Tree*)
- **HashMap** (with an unspecified collision-handling approach)

(a) Compare and contrast HashSet, ListSet, and HashMap data structures with respect to:

- Ordering and uniqueness behaviour
- Average & worst-case time complexity for insertion, lookup & deletion
- Memory overhead

Use examples where appropriate.

(24 marks)

KEY ASPECTS:

Ordering and Uniqueness Behaviour:

	Ordering	**Uniqueness**
HashSet	Unordered — iteration order is unpredictable.	Automatically prevents duplicates; relies on hashCode() and equals() methods.

Marking Scheme

ListSet	Can be maintained in ordered, but this requires special implementation logic. Standard: unordered	No guarantee of uniqueness – duplicate management must be included through specific implementation logic
HashMap	Unordered.	Keys must be unique; duplicate keys overwrite existing entries. Values may repeat.

Key contrast points:

- Only ListSet can provide sorted order if managed correctly; others are unordered.
- Hash-based structures enforce uniqueness of stored keys/elements, List-based structure requires manual management in addition to standard list logic.
- HashMap manages key–value pairs; duplicates are determined by key only.

Average & Worst-Case Time Complexity for Insertion, Lookup & Deletion:

	HashSet	**ListSet**	**HashMap**
Insertion	Average: O(1); Worst: O(n) if many collisions	In sorted order: Average/Worst: O(n) In unsorted order: Average: O(1) Worst: O(n)	Average: O(1); Worst: O(n) if many collisions
Lookup	Average: O(1); Worst: O(n)	Average: O(n); Worst: O(n)	Average: O(1); Worst: O(n)
Deletion	Average: O(1); Worst: O(n)	Average: O(n); Worst: O(n)	Average: O(1); Worst: O(n)

Key contrast points:

Marking Scheme

- HashSet and HashMap have better average-case performance (O(1)) but rely on good hashing.
- ListSet (BST) can give predictable ordering at the expense of O(n) insertions, but must perform O(n) to avoid duplicate data.
- HashMap and HashSet can also degrade to O(n) under excessive hash collisions.

Memory Overhead:

	Memory considerations	**Explanation**
HashSet	Higher due to backing HashMap, extra empty buckets, and load factor management.	Requires reallocation/resizing when load factor exceeded.
ListSet	Each element stored in an array, low overhead spread across all elements as no specific per-element expense Requires extra memory when expanding array to grow capacity	"Purchase"/allocation of larger memory quantity at initialisation that dataset can "grow into". Each expansion requires another allocation of larger space
HashMap	Highest among the three due to key–value pairs plus buckets and possible linked lists or trees for collision handling.	Efficiency depends on load factor and initial capacity settings.

Key contrast points:

- Hash-based structures use extra array capacity to reduce collisions.
- Array-based structures decrease per-object cost, but require extra memory when expanding to fit more elements
- For large datasets, memory tuning (e.g., load factor in HashMap) matters more than raw structure type.

MARKS BREAKDOWN:

8 marks * 3 - Comparison and contrast under heading

Marking Scheme

(b) Recommend **the most suitable data structure** for this analytics platform scenario. Justify your choice by linking its key features to the scenario's requirements, specifically addressing:

- Expected lookup and insertion performance
- The impact of hash collisions
- If the need for sorted order justifies any performance trade-offs

(6 marks)

MARKS BREAKDOWN:

1 mark	-	Verdict:	1 mark
2 marks	-	Justification with regard to lookup and insertion	
2 marks	-	Justification regarding hash collision	
1 mark	-	Justification regarding sorted order justifying performance cost	

Scope for allowable answers is broad. Acceptance is based on justification provided.

Marking Scheme